



SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)

Constituent of Symbiosis International (Deemed University), Pune

(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)

Re-Accredited by NAAC with 'A++' Grade

Name of Student: Aryan Singh Kasera

PRN: 25070122052

Subject: Programming with Java

Sem: III AY: 2026-27

Assignment No: 14

TITLE: Write a Java program to demonstrate exception handling using try and catch blocks, nested and multiple try-catch blocks.

Problem Statement

Write a Java program to demonstrate exception handling using try and catch blocks, nested and multiple try-catch blocks.

Learning Objectives

To handle runtime exceptions using try-catch, nested try, and multiple catch blocks.

Theory

Exception handling prevents abnormal termination of programs due to runtime errors. The try block contains code that may generate exceptions, while catch blocks handle specific exceptions. Nested and multiple try-catch structures allow complex error management.

Exception handling improves software reliability and user experience by gracefully managing unexpected situations.

Create an ATM program handling invalid withdrawal amount using try-catch

ATM.java



Share

Run

```
1 import java.util.Scanner;
2
3 public class ATM {
4     public static void main(String[] args) {
5
6         Scanner sc = new Scanner(System.in);
7
8         int balance = 10000;
9
10        try {
11            System.out.print("Enter withdrawal amount: ");
12            int amount = sc.nextInt();
13
14            if (amount <= 0) {
15                throw new Exception("Withdrawal amount must be greater than zero
16                .");
17            }
18
19            if (amount > balance) {
20                throw new Exception("Insufficient balance.");
21            }
22
23            balance = balance - amount;
24
25            System.out.println("Withdrawal Successful!");
26            System.out.println("Remaining Balance: " + balance);
27        } catch (Exception e) {
28            System.out.println("Error: " + e.getMessage());
29        }
30
31        sc.close();
32    }
33 }
```

OUTPUT

```
Output
Enter withdrawal amount: 12000
ERROR!
Error: Insufficient balance.

=== Code Execution Successful ===
```

Create an Online Shopping program that handles an invalid product quantity entered by the user using a try-catch block. Display an appropriate error message if the quantity is less than or equal to zero

```
OnlineShopping.java
1 import java.util.Scanner;
2
3 public class OnlineShopping {
4     public static void main(String[] args) {
5
6         Scanner sc = new Scanner(System.in);
7
8         try {
9             System.out.print("Enter product quantity: ");
10            int quantity = sc.nextInt();
11
12            if (quantity <= 0) {
13                throw new Exception("Invalid quantity! Quantity must be greater
14                    than zero.");
15            }
16
17            System.out.println("Order placed successfully.");
18            System.out.println("Quantity Ordered: " + quantity);
19        } catch (Exception e) {
20            System.out.println("Error: " + e.getMessage());
21        }
22
23        sc.close();
24    }
25 }
```

OUTPUT

Output
Enter product quantity: 3 Order placed successfully. Quantity Ordered: 3 === Code Execution Successful ===

Conclusion

The program successfully demonstrates exception handling in Java using **try**, **catch**, and **throw** statements. It shows how runtime errors can be detected and handled without terminating the program abruptly. This experiment highlights the importance of exception handling in making Java applications more reliable, user-friendly, and capable of managing unexpected errors effectively.